notify_stock_price.py — 코드만 (8주차)

설명(주석)을 걷어내고 코드 줄만 남긴 것입니다. 구조를 한눈에 보라고 만든 자료이고,
같은 폴더의 notify_stock_price.py 이 원본입니다 — 주석은 그 파일에 그대로 있습니다.
한 줄이 길어 넘어간 곳은 다음 줄에 안쪽으로 들여 이어 붙였습니다.
이모지(✈ ✈ 🌐 🌐 …)는 이 PDF 글꼴에 없어 네모로 보입니다 — 원본에는 정상입니다.
※ 이번 주에 새로 생긴 줄이 없습니다 — bigd_07_chart 와 코드가 같습니다.
==

```
import os
import io
import re
import time
import html
import requests
from datetime import datetime
from zoneinfo import ZoneInfo
from dotenv import load_dotenv


WATCHLIST_FILE = "watchlist.csv"
KST = ZoneInfo("Asia/Seoul")

load_dotenv()

TELEGRAM_BOT_TOKEN = os.environ.get("TELEGRAM_BOT_TOKEN")
TELEGRAM_CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID")


def read_watchlist(path: str = WATCHLIST_FILE) -> list:
    if not os.path.exists(path):
        print(f"✈ '{path}' 파일이 존재하지 않습니다.")
        return None

    try:
        import pandas as pd

        watchlist_df = pd.read_csv(path, dtype={"code": str})

        watchlist_df.columns = [col.strip().lower() for col in watchlist_df.columns]

        if "code" not in watchlist_df.columns:
            print("✈ CSV 파일에 'code' 컬럼이 존재하지 않습니다.")
            return None

        codes = watchlist_df["code"].tolist()
        print(f"🌐 읽어온 관심 종목 리스트: {codes}")
        return codes

    except Exception as e:
        print(f"✈ CSV 파일을 읽는 동안 오류가 발생했습니다: {e}")
        return None


def fetch_naver_current_price(code: str, retries: int = 2) -> dict:
    url = f"https://m.stock.naver.com/api/stock/{code}/basic"
    for attempt in range(1, retries + 1):
        try:
```

```
            response = requests.get(url, headers={"User-Agent": "Mozilla/5.0"},
                    timeout=3)
            if response.status_code != 200:
                print(f"⚠ [{code}] 네이버 현재가 조회 실패 (응답 코드: {response.status_code},
                        {attempt}/{retries}번째 시도)")
            else:
                data = response.json()
                price_str = data.get("closePrice", "0").replace(",", "")
                price = int(price_str) if price_str.isdigit() else 0
                if price <= 0:
                    print(f"⚠ [{code}] 가격을 숫자로 읽지 못했습니다 "
                          f"(closePrice: {data.get('closePrice')!r},
                                  {attempt}/{retries}번째 시도)")
                else:
                    return {
                        "code": code,
                        "name": data.get("stockName", "알 수 없음"),
                        "price": price,
                        "rate": float(data.get("fluctuationsRatio", "0") or "0"),
                        "is_open": data.get("marketStatus") == "OPEN",
                        "traded_at": data.get("localTradedAt"),
                    }
        except Exception as e:
            print(f"⚠ [{code}] 네이버 현재가 조회 중 오류 발생: {e} ({attempt}/{retries}번째 시도)")

        if attempt < retries:
            time.sleep(1)

    return None


def fetch_naver_intraday_minutes(code: str, date_str: str = None) -> list:
    date_str = date_str or datetime.now(KST).strftime("%Y%m%d")
    url = "https://api.finance.naver.com/siseJson.naver"
    params = {"symbol": code, "requestType": 1, "startTime": date_str, "endTime":
            date_str, "timeframe": "minute"}
    try:
        response = requests.get(url, params=params, headers={"User-Agent":
                "Mozilla/5.0"}, timeout=3)
        if response.status_code != 200:
            print(f"⚠ [{code}] 당일 분봉 조회 실패 (응답 코드: {response.status_code})")
            return []
        rows =
                re.findall(r'\["(\d{12})",\s*null,\s*null,\s*null,\s*(\d+),\s*\d+,\s*nul
                l\]', response.text)
        if not rows and re.search(r'\["\d{12}"', response.text):
            print(f"⚠[{code}]  당일 분봉 응답 형식이 예상과 달라 파싱하지 못했습니다. "
                  f"네이버 API 응답 형식이 바뀌었을 수 있습니다.")
        minutes = [{"time": timestamp[8:], "price": int(price)} for timestamp, price in
                rows]
        minutes.reverse()
        return minutes
    except Exception as e:
        print(f"⚠ [{code}] 당일 분봉 조회 중 오류 발생: {e}")
        return []


def send_telegram_message(text: str) -> bool:
```

```
    url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
    data = {"chat_id": TELEGRAM_CHAT_ID, "text": text, "parse_mode": "HTML"}
    try:
        response = requests.post(url, data=data, timeout=15)
        return response.status_code == 200
    except Exception as e:
        print(f"✈ 텔레그램 메시지 전송 중 오류 발생: {e}")
        return False


def send_telegram_photo(photo_buffer: io.BytesIO, caption: str) -> bool:
    url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendPhoto"
    files = {"photo": ("chart.png", photo_buffer, "image/png")}
    data = {"chat_id": TELEGRAM_CHAT_ID, "caption": caption, "parse_mode": "HTML"}
    try:
        response = requests.post(url, data=data, files=files, timeout=15)
        return response.status_code == 200
    except Exception as e:
        print(f"✈ 텔레그램 이미지 전송 중 오류 발생: {e}")
        return False


def describe_price_trend(daily_closes: list, intraday_minutes: list) -> str:
    if daily_closes:
        return f"최근 {len(daily_closes)}일 종가 + 현재가 추이"
    if intraday_minutes:
        return "오늘 현재가 추이 (종가 히스토리 누적 전)"
    return "현재가 (종가 히스토리 누적 전)"


def format_rate_badge(price: int, rate: float) -> str:
    prefix = f"{price:,}원"
    if rate > 0:
        return f"{prefix} 🈴 +{rate}%"
    if rate < 0:
        return f"{prefix} ▼ {rate}%"
    return f"{prefix} ▫0.0%"


def dedupe_daily_closes(daily_closes: list, today_price: int, intraday_minutes: list)
        -> list:
    if today_price is None and not intraday_minutes:
        return daily_closes
    today_str = datetime.now(KST).strftime("%Y%m%d")
    return [entry for entry in daily_closes if entry["date"] != today_str]


def _evenly_spaced(start: float, end: float, n: int) -> list:
    if n <= 0:
        return []
    if n == 1:
        return [start]
    step = (end - start) / (n - 1)
    return [start + step * i for i in range(n)]


def build_price_chart(daily_closes: list, current_price: int = None,
                      intraday_minutes: list = None) -> io.BytesIO:
```

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager

installed_fonts = {f.name for f in font_manager.fontManager.ttflist}
korean_fonts = [name for name in ("Malgun Gothic", "NanumGothic", "AppleGothic")
                if name in installed_fonts]
plt.rcParams["font.family"] = korean_fonts or ["sans-serif"]
plt.rcParams["axes.unicode_minus"] = False

if intraday_minutes:
    today_prices = [m["price"] for m in intraday_minutes]
elif current_price is not None:
    today_prices = [current_price]
else:
    today_prices = []

daily_closes = dedupe_daily_closes(daily_closes, current_price, intraday_minutes)

daily_labels = [f"{entry['date'][4:6]}/{entry['date'][6:]}" for entry in
        daily_closes]
daily_prices = [entry["close"] for entry in daily_closes]

prices = daily_prices + today_prices

if intraday_minutes:
    daily_x = _evenly_spaced(0.0, 0.5, len(daily_prices))
    today_x = _evenly_spaced(0.5, 1.0, len(today_prices))
    x = daily_x + today_x
else:
    x = list(range(len(prices)))
    daily_x = x[:len(daily_prices)]

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(x, prices, color="#1f77b4", linewidth=1.5)

if daily_x:
    ax.plot(daily_x, daily_prices, marker="o", linestyle="None", color="#1f77b4")
    for xi, yi in zip(daily_x, daily_prices):
        ax.annotate(f"{yi:,}", (xi, yi), textcoords="offset points", xytext=(0, 8),
                ha="center", fontsize=13)

if today_prices:
    ax.plot(x[-1], prices[-1], marker="o", color="#d62728")
    ax.annotate(f"{prices[-1]:,}", (x[-1], prices[-1]), textcoords="offset points",
                xytext=(0, 8), ha="center", fontsize=13, color="#d62728")

tick_positions = list(daily_x)
tick_labels = list(daily_labels)
if intraday_minutes:
    for xi, m in zip(today_x, intraday_minutes):
        if m["time"].endswith("00"):
            tick_positions.append(xi)
            tick_labels.append(f"{m['time'][:2]}:{m['time'][2:]}")
elif current_price is not None:
    tick_positions.append(x[-1])
    tick_labels.append("현재")
```

```
    ax.set_xticks(tick_positions)
    ax.set_xticklabels(tick_labels, rotation=45, ha="right", fontsize=8)

    ax.set_ylabel("가격 (원)")
    ax.grid(True, alpha=0.3)
    fig.tight_layout()

    buffer = io.BytesIO()
    fig.savefig(buffer, format="png")
    plt.close(fig)
    buffer.seek(0)
    return buffer


def send_price_notification() -> bool:
    watchlist_codes = read_watchlist()
    if watchlist_codes is None:
        return False

    print("📡 관심종목 현재가 조회 시작...")

    current_prices = {}
    telegram_message = "🌐 <b>내 관심종목 현재가</b>"

    for code in watchlist_codes:
        info = fetch_naver_current_price(code)
        if info is None:
            continue

        current_prices[code] = info

    if not current_prices:
        print("✈️ 관심종목의 현재가를 하나도 가져오지 못했습니다. 네이버 API 상태를 확인해 주세요.")
        return False

    intraday_by_code = {code: fetch_naver_intraday_minutes(code) for code in
            current_prices}

    first_code = next(iter(current_prices))
    first_intraday_minutes = intraday_by_code[first_code]
    telegram_message += f"\n\n{describe_price_trend([], first_intraday_minutes)}"

    sent = 0

    if send_telegram_message(telegram_message):
        sent += 1
        print("✈️ 현재가 메시지를 텔레그램으로 전송했습니다!")
    else:
        print("✈️ 현재가 메시지 전송에 실패했습니다.")

    for code, info in current_prices.items():
        intraday_minutes = intraday_by_code[code]
        chart_buffer = build_price_chart([], info["price"], intraday_minutes)
        caption = (
            f"🌐 <b>{html.escape(info['name'])}</b> ({code})\n"
            f"    {format_rate_badge(info['price'], info['rate'])}"
        )
```

```
        if send_telegram_photo(chart_buffer, caption):
            sent += 1
            print(f"📊 [{code}] 추이 그래프를 텔레그램으로 전송했습니다!")
        else:
            print(f"📱 [{code}] 추이 그래프 전송에 실패했습니다.")

    if sent == 0:
        print("📱 텔레그램으로 아무것도 전송하지 못했습니다. 봇 토큰과 텔레그램 API 상태를 확인해 주세요.")
        return False

    return True


if __name__ == "__main__":
    send_price_notification()
```